implementation_demo

May 6, 2025

1 LM Watermarking implementation Demo

This notebook is heavily based on the work here https://github.com/jthickstun/watermark/

The notebook was executed on a google colab project online.

[1]:
```
# Notebook parameters (modify as needed)#
#model_name = "facebook/opt-2.7b"
model_name = "huggyllama/llama-7b"
prompt = "Steganography is"
m = 100             # length of generation
n = 256            # watermark sequence length
key = 42           # watermark key
seed = 1           # random seed for reproducibility
```

[2]:
```
import torch
import numpy as np
import random
import tqdm
from transformers import AutoTokenizer, AutoModelForCausalLM

device = torch.device(
    'cuda' if torch.cuda.is_available() else
    ('mps' if torch.backends.mps.is_available() else 'cpu')
)
print(device)
```

```
cpu
```

[3]:
```
# Seed for reproducibility
torch.manual_seed(seed)
np.random.seed(seed)
random.seed(seed)

# Initialize tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
```

```
    device_map="auto",
)

# Prepare inputs
inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
You are using the default legacy behaviour of the <class
'transformers.models.llama.tokenization_llama_fast.LlamaTokenizerFast'>. This is
expected, and simply means that the `legacy` (previous) behavior will be used so
nothing changes for you. If you want to use the new behaviour, set
`legacy=False`. This should only be set if you understand what it means, and
thoroughly read the reason why this was added as explained in
https://github.com/huggingface/transformers/pull/24565 - if you loaded a llama
tokenizer from a GGUF file you can ignore this message.

Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]
```

[4]:
```
# Generate normally
outputs = model.generate(
    inputs,
    max_new_tokens=m,
    do_sample=True,
)

# Decode and display
normal_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("--- Normal Output ---")
print(normal_text)
```

```
--- Normal Output ---
Steganography is a technique used to hide secret messages within an image, a
song, or other media that are not usually used to convey secure information.
Steganography can be used to conceal the fact that there is a secret message in
the first place, but it can also be used to transmit secret messages. There are
many different methods of encoding secret messages in an image. Some of the
methods are:
Compression method (also called Spread spectrum)
The key challenge with steganography is embedding
```

[5]:
```python
# Mersenne Twister based RNG for watermarking
def int_32(x): return int(0xFFFFFFFF & x)

class mersenne_rng:
    def __init__(self, seed=5489):
        self.state = [0]*624
        self.index = 624
        self.upper_mask = (1<<31)
        self.lower_mask = (1<<31) - 1
        # constants
        self.f, self.m = 1812433253, 397
        self.u, self.s, self.b = 11, 7, 0x9D2C5680
        self.t, self.c, self.l = 15, 0xEFC60000, 18
        # initialize state
        self.state[0] = seed
        for i in range(1, 624):
            prev = self.state[i-1]
            self.state[i] = int_32(self.f * (prev ^ (prev >> 30)) + i)

    def twist(self):
        for i in range(624):
            x = (self.state[i] & self.upper_mask) + (self.state[(i+1)%624] &␣
 ↪self.lower_mask)
            xA = x >> 1
            if x % 2: xA ^= 0x9908b0df
            self.state[i] = self.state[(i + self.m) % 624] ^ xA
        self.index = 0

    def rand(self):
        if self.index >= 624:
            self.twist()
        y = self.state[self.index]
        y ^= (y >> self.u)
        y ^= ((y << self.s) & self.b)
        y ^= ((y << self.t) & self.c)
        y ^= (y >> self.l)
        self.index += 1
        return (int_32(y) * (1.0 / 4294967296.0))
```

[6]:
```python
# Exponential watermark sampling
def exp_sampling(probs, u):
    # probs: tensor [vocab], u: numpy array [vocab]
    return torch.argmax(torch.from_numpy(u) ** (1.0 / probs.cpu().numpy()),␣
 ↪dim=1).unsqueeze(-1)

def generate_shift(model, prompt_ids, vocab_size, n, m, key):
    rng = mersenne_rng(key)
```

```
    # create watermark matrix xi
    xi = np.array([rng.rand() for _ in range(n * vocab_size)], dtype=np.
↪float32).reshape(n, vocab_size)
    torch.manual_seed(seed)
    shift = torch.randint(0, n, (1,)).item()
    print(f"Shift: {shift}")

    inputs = prompt_ids.to(model.device)
    attn = torch.ones_like(inputs)
    past = None
    for i in tqdm.tqdm(range(m)):
        with torch.no_grad():
            if past:
                out = model(inputs[:, -1:], past_key_values=past,␣
↪attention_mask=attn)
            else:
                out = model(inputs)
        probs = torch.nn.functional.softmax(out.logits[:, -1, :vocab_size],␣
↪dim=-1)
        u = xi[(shift + i) % n]
        token = exp_sampling(probs, u)
        inputs = torch.cat([inputs, token.to(model.device)], dim=-1)
        past = out.past_key_values
        attn = torch.cat([attn, torch.ones((attn.shape[0],1), device=attn.
↪device)], dim=-1)

    return inputs.cpu()
```

[7]:
```
# Encode prompt & run watermark generation
tokens = tokenizer.encode(prompt, return_tensors='pt')
wm_tokens = generate_shift(model, tokens, len(tokenizer), n, m, key)[0]
watermarked_text = tokenizer.decode(wm_tokens, skip_special_tokens=True)
print()
print("--- Watermarked Output ---")
print(watermarked_text)
```

```
Shift: 37

  0%|          | 0/100 [00:00<?, ?it/s]<ipython-input-6-090363e8b445>:4:
DeprecationWarning: __array_wrap__ must accept context and return_scalar
arguments (positionally) in the future. (Deprecated NumPy 2.0)
  return torch.argmax(torch.from_numpy(u) ** (1.0 / probs.cpu().numpy()),
dim=1).unsqueeze(-1)
100%|     | 100/100 [01:18<00:00,  1.28it/s]


--- Watermarked Output ---
Steganography is the practice of hiding information or data inside another file
```

or software even if a malicious party tries their hardest to reveal the hidden codes. It basically closes the door to hackers and only allotts access to those who are suppose to take it. This makes it, in a sense, lie to the user and or viewer of the data and anyone trying to open the file.
Stegano is a useful tool for exchanging messages without anyone but the sender and receiver

[8]:
```python
from numba import njit, prange
import math

@njit
def levenshtein(x, y, gamma=0.0):
    n = len(x)
    m = len(y)
    A = np.zeros((n + 1, m + 1), dtype=np.float64)
    for i in range(n + 1):
        for j in range(m + 1):
            if i == 0:
                A[i, j] = j * gamma
            elif j == 0:
                A[i, j] = i * gamma
            else:
                cost_sub = math.log(1.0 - y[j - 1, x[i - 1]])
                # deletion
                best = A[i - 1, j] + gamma
                # insertion
                tmp = A[i, j - 1] + gamma
                if tmp < best:
                    best = tmp
                # substitution
                tmp = A[i - 1, j - 1] + cost_sub
                if tmp < best:
                    best = tmp
                A[i, j] = best
    return A[n, m]
```

[9]:
```python
import time

from numba import njit, prange


@njit
def detect(tokens, n, k, xi, gamma=0.0):
    m = len(tokens)
    n = len(xi)
```

```
    A = np.empty((m - (k - 1), n))
    for i in range(m - (k - 1)):
        for j in range(n):
            A[i][j] = levenshtein(tokens[i : i + k], xi[(j + np.arange(k)) %␣
 ↪n], gamma)

    return np.min(A)

def permutation_test(tokens, key, n, k, vocab_size, n_runs=100):
    rng = mersenne_rng(key)
    xi = np.array(
        [rng.rand() for _ in range(n * vocab_size)], dtype=np.float32
    ).reshape(n, vocab_size)
    test_result = detect(tokens, n, k, xi)

    p_val = 0
    for run in tqdm.tqdm(range(n_runs)):
        xi_alternative = np.random.rand(n, vocab_size).astype(np.float32)
        null_result = detect(tokens, n, k, xi_alternative)

        # assuming lower test values indicate presence of watermark
        p_val += null_result <= test_result

    return (p_val + 1.0) / (n_runs + 1.0)


# Run decoding test
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokens = tokenizer.encode(watermarked_text, return_tensors='pt',␣
 ↪truncation=True, max_length=2048).numpy()[0]
tok_list = tokens.tolist()
print(tok_list)
start = time.time()
p_val_watermarked = permutation_test(tok_list, key, n, len(tok_list),␣
 ↪len(tokenizer))
print()
print(f"p-value: {p_val_watermarked:.4f} (elapsed {time.time()-start:.1f}s)")
print(f"Is watermarked: {p_val_watermarked<0.01}")
```

```
[1, 2443, 6249, 5275, 338, 278, 6944, 310, 25508, 2472, 470, 848, 2768, 1790,
934, 470, 7047, 1584, 565, 263, 4439, 14803, 6263, 14335, 1009, 2898, 342, 304,
10320, 284, 278, 7934, 11561, 29889, 739, 8830, 4694, 267, 278, 3050, 304,
15833, 414, 322, 871, 599, 327, 1372, 2130, 304, 1906, 1058, 526, 7755, 304,
2125, 372, 29889, 910, 3732, 372, 29892, 297, 263, 4060, 29892, 3804, 304, 278,
1404, 322, 470, 6316, 556, 310, 278, 848, 322, 5019, 1811, 304, 1722, 278, 934,
29889, 13, 7789, 29887, 1562, 338, 263, 5407, 5780, 363, 429, 305, 9776, 7191,
1728, 5019, 541, 278, 10004, 322, 19870]
```

```
100%|      | 100/100 [01:24<00:00,  1.18it/s]

p-value: 0.0099 (elapsed 99.3s)
Is watermarked: True
```

[10]:
```
!pip install googletrans==4.0.0-rc1
from googletrans import Translator

def back_translate(text, target_language='es'):
    """Translates text to a target language and back to the original language.
    """
    translator = Translator()
    translated = translator.translate(text, dest=target_language).text
    back_translated = translator.translate(translated, dest='en').text
    return back_translated

#def add_noise(text, noise_level=0.1):
#    """Adds random noise to the text."""
#    words = text.split()
#    num_noise_words = int(len(words) * noise_level)
#    noise_words = ["noise" for _ in range(num_noise_words)]
#    for i in range(num_noise_words):
#        words.insert(random.randint(0, len(words)), noise_words[i])
#    return " ".join(words)

# Apply back-translation to the watermarked text EN -> FR -> EN -> RU -> EN
back_translated_text = back_translate(back_translate(watermarked_text, "fr"),␣
    ↪"ru")
print("Back translated: ", back_translated_text)

# Encode the back-translated text
back_translated_tokens = tokenizer.encode(back_translated_text,␣
    ↪return_tensors='pt', truncation=True, max_length=2048).numpy()[0]
back_translated_tok_list = back_translated_tokens.tolist()

# Detect watermark in the back-translated text
start = time.time()
p_val_back_translated = permutation_test(back_translated_tok_list, key, n,␣
    ↪len(back_translated_tok_list), len(tokenizer))
print()
print(f"p-value (back-translated): {p_val_back_translated:.4f} (elapsed {time.
    ↪time()-start:.1f}s)")
print(f"Is watermarked (back-translated): {p_val_back_translated<0.01}")
```

```
Requirement already satisfied: googletrans==4.0.0-rc1 in
/usr/local/lib/python3.11/dist-packages (4.0.0rc1)
```

```
Requirement already satisfied: httpx==0.13.3 in /usr/local/lib/python3.11/dist-
packages (from googletrans==4.0.0-rc1) (0.13.3)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2025.4.26)
Requirement already satisfied: hstspreload in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2025.1.1)
Requirement already satisfied: sniffio in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (1.3.1)
Requirement already satisfied: chardet==3.* in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (3.0.4)
Requirement already satisfied: idna==2.* in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2.10)
Requirement already satisfied: rfc3986<2,>=1.3 in
/usr/local/lib/python3.11/dist-packages (from
httpx==0.13.3->googletrans==4.0.0-rc1) (1.5.0)
Requirement already satisfied: httpcore==0.9.* in
/usr/local/lib/python3.11/dist-packages (from
httpx==0.13.3->googletrans==4.0.0-rc1) (0.9.1)
Requirement already satisfied: h11<0.10,>=0.8 in /usr/local/lib/python3.11/dist-
packages (from httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (0.9.0)
Requirement already satisfied: h2==3.* in /usr/local/lib/python3.11/dist-
packages (from httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (3.2.0)
Requirement already satisfied: hyperframe<6,>=5.2.0 in
/usr/local/lib/python3.11/dist-packages (from
h2==3.*->httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (5.2.0)
Requirement already satisfied: hpack<4,>=3.0 in /usr/local/lib/python3.11/dist-
packages (from h2==3.*->httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1)
(3.0.0)
Back translated:  Steganography is the practice of hiding information or data in
another file or software, even if the malicious side tries to open hidden codes.
In essence, it closes the pirate door and attributes only access to those who
should take it. This is, in a sense, to lie to the user and / or a data viewer
and anyone who tries to open the file.
Stegano - a useful tool for exchanging messages without anyone, except the
sender and recipient

100%|      | 100/100 [01:22<00:00,  1.21it/s]


p-value (back-translated): 0.0099 (elapsed 94.7s)
Is watermarked (back-translated): True
```

[11]:
```
tokens_control = tokenizer.encode(normal_text, return_tensors='pt',␣
 ↪truncation=True, max_length=2048).numpy()[0]
tok_control_list = tokens_control.tolist()
print(tok_control_list)
start = time.time()
```

```
p_val_control = permutation_test(tok_control_list, key, n,␣
 ↪len(tok_control_list), len(tokenizer))
print()
print(f"p-value (control): {p_val_control:.4f} (elapsed {time.time()-start:.
 ↪1f}s)")
print(f"Is watermarked (control): {p_val_control<0.01}")
```

```
[1, 2443, 6249, 5275, 338, 263, 11043, 1304, 304, 9563, 7035, 7191, 2629, 385,
1967, 29892, 263, 4823, 29892, 470, 916, 5745, 393, 526, 451, 5491, 1304, 304,
27769, 11592, 2472, 29889, 2443, 6249, 5275, 508, 367, 1304, 304, 10628, 284,
278, 2114, 393, 727, 338, 263, 7035, 2643, 297, 278, 937, 2058, 29892, 541, 372,
508, 884, 367, 1304, 304, 22649, 7035, 7191, 29889, 1670, 526, 1784, 1422, 3519,
310, 8025, 7035, 7191, 297, 385, 1967, 29889, 3834, 310, 278, 3519, 526, 29901,
13, 1523, 2590, 1158, 313, 15189, 2000, 1706, 949, 18272, 29897, 13, 1576, 1820,
18766, 411, 1886, 6249, 5275, 338, 23655]
```

```
100%|      | 100/100 [01:35<00:00,  1.05it/s]


p-value (control): 0.8020 (elapsed 107.8s)
Is watermarked (control): False
```

[12]:
```
print(f"""
--- Plain/Control text (p={p_val_control:.4f}) ---
{normal_text}

--- Watermarked (p={p_val_watermarked:.4f}) ---
{watermarked_text}

--- Edited (p={p_val_back_translated:.4f}) ---
{back_translated_text}
""")
```

```
--- Plain/Control text (p=0.8020) ---
Steganography is a technique used to hide secret messages within an image, a
song, or other media that are not usually used to convey secure information.
Steganography can be used to conceal the fact that there is a secret message in
the first place, but it can also be used to transmit secret messages. There are
many different methods of encoding secret messages in an image. Some of the
methods are:
Compression method (also called Spread spectrum)
The key challenge with steganography is embedding

--- Watermarked (p=0.0099) ---
Steganography is the practice of hiding information or data inside another file
or software even if a malicious party tries their hardest to reveal the hidden
codes. It basically closes the door to hackers and only allotts access to those
```

```
who are suppose to take it. This makes it, in a sense, lie to the user and or
viewer of the data and anyone trying to open the file.
Stegano is a useful tool for exchanging messages without anyone but the sender
and receiver

--- Edited (p=0.0099) ---
Steganography is the practice of hiding information or data in another file or
software, even if the malicious side tries to open hidden codes. In essence, it
closes the pirate door and attributes only access to those who should take it.
This is, in a sense, to lie to the user and / or a data viewer and anyone who
tries to open the file.
Stegano - a useful tool for exchanging messages without anyone, except the
sender and recipient
```

1.1 Verbose Generation

Really go through the process step by step to see how each of the probabilites get affected.

[16]:
```python
def verbose_generate(model, prompt, n=256, m=20, key=42, top_k=10):
    """
    Generate *m* watermarked tokens from *prompt* while printing rich␣
 ↪diagnostics.
    """

    tokenizer  = AutoTokenizer.from_pretrained(model_name)
    vocab_size = len(tokenizer)

    rng = mersenne_rng(key)
    xi  = np.array([rng.rand() for _ in range(n * vocab_size)],
                   dtype=np.float32).reshape(n, vocab_size)
    torch.manual_seed(seed)
    shift = torch.randint(0, n, (1,)).item()
    print(f"Shift: {shift}")

    inputs = tokenizer.encode(prompt, return_tensors="pt").to(model.device)
    attn   = torch.ones_like(inputs)
    past   = None

    print(f"Initial prompt: {prompt!r}\n\n---  BEGIN VERBOSE GENERATION  ---")

    for i in range(m):
        with torch.no_grad():
            out = model(
                inputs if past is None else inputs[:, -1:],
                past_key_values=past,
                attention_mask=attn,
            )
```

```
            lm_probs = torch.softmax(out.logits[:, -1, :vocab_size], dim=-1)[0]

        j   = (shift + i) % n
        row = torch.from_numpy(xi[j]).to(lm_probs.device)

        wm_weights = torch.pow(row, 1.0 / lm_probs)
        wm_probs   = wm_weights / wm_weights.sum()

        top_idx = torch.topk(lm_probs, k=top_k).indices.tolist()

        print(f"\nStep {i:02d} - j = {j}")
        print(" token                          p_LM     xi      p_WM")
        print("                               ")
        for t in top_idx:
            token_str = tokenizer.decode([t]).replace("\n", "\\n")
            print(f"{token_str:<26}  {lm_probs[t]:.4f}  {row[t]:.4f}
↪{wm_probs[t]:.4f}")

        token = exp_sampling(lm_probs.unsqueeze(0), row.cpu().numpy()).item()
        chosen = tokenizer.decode([token]).replace("\n", "\\n")
        print(f" → Chosen token: {token:>4d}  ('{chosen}')")

        inputs = torch.cat([inputs, torch.tensor([[token]], device=inputs.
↪device)], dim=-1)
        past   = out.past_key_values
        attn   = torch.cat([attn, torch.ones((attn.shape[0], 1), device=attn.
↪device)], dim=-1)

        current_text = tokenizer.decode(inputs[0], skip_special_tokens=True)
        print(" Current prompt so far →", repr(current_text))

    print("---  END VERBOSE GENERATION  ---\n")
    return tokenizer.decode(inputs[0], skip_special_tokens=True)

verbose_text = verbose_generate(model, prompt, n=n, m=20, key=key)
print("\nFinal partial generation:\n", verbose_text)
```

```
Shift: 37
Initial prompt: 'Steganography is'

---  BEGIN VERBOSE GENERATION  ---

Step 00 - j = 37
 token                          p_LM     xi      p_WM

the                           0.6179  0.6642  0.9988
a                             0.1936  0.2224  0.0008
```

```
an                          0.0343  0.4728  0.0000
one                         0.0095  0.1159  0.0000
hiding                      0.0080  0.2565  0.0000
not                         0.0069  0.6270  0.0000
defined                     0.0057  0.4347  0.0000
used                        0.0051  0.1604  0.0000
basically                   0.0039  0.3203  0.0000
often                       0.0034  0.1888  0.0000
 → Chosen token:  278  ('the')
 Current prompt so far → 'Steganography is the'

<ipython-input-6-090363e8b445>:4: DeprecationWarning: __array_wrap__ must accept
context and return_scalar arguments (positionally) in the future. (Deprecated
NumPy 2.0)
  return torch.argmax(torch.from_numpy(u) ** (1.0 / probs.cpu().numpy()),
dim=1).unsqueeze(-1)


Step 01 - j = 38
 token                        p_LM     xi      p_WM

art                         0.4627  0.2408  0.0556
practice                    0.1638  0.9550  0.9110
process                     0.0665  0.1585  0.0000
science                     0.0655  0.7853  0.0302
technique                   0.0316  0.0601  0.0000
study                       0.0199  0.1586  0.0000
hiding                      0.0197  0.1910  0.0000
act                         0.0186  0.2022  0.0000
method                      0.0124  0.1854  0.0000
name                        0.0093  0.7448  0.0000
 → Chosen token: 6944  ('practice')
 Current prompt so far → 'Steganography is the practice'

Step 02 - j = 39
 token                        p_LM     xi      p_WM

of                          0.9547  0.4524  0.9978
and                         0.0235  0.3652  0.0000
or                          0.0077  0.2826  0.0000
,                           0.0040  0.7004  0.0000
that                        0.0019  0.9407  0.0000
to                          0.0012  0.2799  0.0000
by                          0.0008  0.0994  0.0000
in                          0.0007  0.8650  0.0000
used                        0.0006  0.1539  0.0000
where                       0.0005  0.2094  0.0000
 → Chosen token:  310  ('of')
 Current prompt so far → 'Steganography is the practice of'
```

```
Step 03 - j = 40
 token                          p_LM     xi      p_WM

hiding                        0.5877  0.2907  0.3969
conce                         0.2393  0.5335  0.2353
embedding                     0.0312  0.4600  0.0000
secret                        0.0145  0.0479  0.0000
writing                       0.0128  0.5202  0.0000
using                         0.0096  0.3912  0.0000
obsc                          0.0084  0.0113  0.0000
encoding                      0.0082  0.1361  0.0000
communic                      0.0081  0.6516  0.0000
encrypt                       0.0079  0.6208  0.0000
 → Chosen token: 25508  ('hiding')
 Current prompt so far → 'Steganography is the practice of hiding'

Step 04 - j = 41
 token                          p_LM     xi      p_WM

messages                      0.2000  0.5555  0.1517
information                   0.1943  0.7710  0.7513
data                          0.1500  0.3428  0.0023
a                             0.1323  0.2594  0.0001
secret                        0.1252  0.1291  0.0000
the                           0.0290  0.8610  0.0165
or                            0.0123  0.8681  0.0000
sensitive                     0.0106  0.2001  0.0000
something                     0.0105  0.7360  0.0000
text                          0.0075  0.8341  0.0000
 → Chosen token: 2472  ('information')
 Current prompt so far → 'Steganography is the practice of hiding information'

Step 05 - j = 42
 token                          p_LM     xi      p_WM

in                            0.4128  0.3204  0.0569
within                        0.2405  0.8240  0.4010
or                            0.0587  0.9707  0.5402
inside                        0.0473  0.3905  0.0000
,                             0.0425  0.3039  0.0000
.                             0.0272  0.1466  0.0000
(                             0.0262  0.1873  0.0000
by                            0.0155  0.3577  0.0000
and                           0.0139  0.4302  0.0000
to                            0.0098  0.4717  0.0000
 → Chosen token:  470  ('or')
 Current prompt so far → 'Steganography is the practice of hiding information
```

```
or'

Step 06 - j = 43
 token                          p_LM     xi      p_WM

messages                      0.4289  0.6481  0.1844
a                             0.1225  0.8668  0.1578
data                          0.0866  0.9805  0.4039
communication                 0.0451  0.9078  0.0595
communic                      0.0392  0.5786  0.0000
files                         0.0365  0.1152  0.0000
message                       0.0228  0.3551  0.0000
an                            0.0167  0.9419  0.0140
secret                        0.0140  0.9098  0.0006
other                         0.0129  0.8953  0.0001
 → Chosen token:  848  ('data')
 Current prompt so far → 'Steganography is the practice of hiding information or
data'

Step 07 - j = 44
 token                          p_LM     xi      p_WM

in                            0.4172  0.3880  0.0890
within                        0.3294  0.5037  0.1073
inside                        0.0450  0.9968  0.8022
by                            0.0322  0.5999  0.0000
into                          0.0249  0.7270  0.0000
,                             0.0169  0.3087  0.0000
.                             0.0123  0.8020  0.0000
(                             0.0118  0.0379  0.0000
so                            0.0101  0.6093  0.0000
from                          0.0075  0.9024  0.0000
 → Chosen token: 2768  ('inside')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside'

Step 08 - j = 45
 token                          p_LM     xi      p_WM

another                       0.3282  0.9122  0.5790
a                             0.1334  0.0519  0.0000
other                         0.1147  0.8349  0.1589
an                            0.1068  0.4628  0.0006
of                            0.0704  0.0595  0.0000
something                     0.0402  0.6814  0.0001
images                        0.0220  0.0028  0.0000
the                           0.0212  0.8157  0.0001
seem                          0.0178  0.4946  0.0000
```

```
innoc                         0.0124  0.3759  0.0000
 → Chosen token: 1790  ('another')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another'

Step 09 - j = 46
 token                          p_LM     xi      p_WM

file                          0.2416  0.7254  0.6968
piece                         0.1735  0.0161  0.0000
object                        0.0766  0.8472  0.3017
message                       0.0727  0.4134  0.0000
data                          0.0664  0.3171  0.0000
form                          0.0549  0.2936  0.0000
medium                        0.0516  0.6087  0.0002
type                          0.0225  0.8214  0.0004
format                        0.0162  0.1777  0.0000
document                      0.0141  0.8934  0.0009
 → Chosen token:  934  ('file')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file'

Step 10 - j = 47
 token                          p_LM     xi      p_WM

.                             0.2530  0.9080  0.2473
,                             0.1886  0.4668  0.0064
or                            0.1714  0.9888  0.3393
in                            0.0369  0.9556  0.1057
format                        0.0366  0.5322  0.0000
(                             0.0353  0.9890  0.2649
so                            0.0343  0.5144  0.0000
such                          0.0311  0.0777  0.0000
that                          0.0296  0.2613  0.0000
and                           0.0273  0.3319  0.0000
 → Chosen token:  470  ('or')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or'

Step 11 - j = 48
 token                          p_LM     xi      p_WM

data                          0.1141  0.0015  0.0000
document                      0.1038  0.3265  0.0000
object                        0.0913  0.5254  0.0007
message                       0.0892  0.2964  0.0000
image                         0.0811  0.2870  0.0000
file                          0.0456  0.1774  0.0000
```

```
a                           0.0432  0.6282  0.0000
stream                      0.0321  0.2360  0.0000
in                          0.0317  0.7812  0.0003
media                       0.0291  0.6347  0.0000
 → Chosen token: 7047  ('software')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software'

Step 12 - j = 49
 token                        p_LM     xi      p_WM

.                           0.3799  0.4944  0.0813
program                     0.0944  0.9283  0.2362
,                           0.0865  0.1092  0.0000
application                 0.0601  0.4776  0.0000
so                          0.0523  0.6839  0.0004
that                        0.0388  0.0989  0.0000
in                          0.0372  0.9299  0.0737
to                          0.0339  0.9705  0.2147
and                         0.0222  0.5515  0.0000
package                     0.0213  0.3590  0.0000
 → Chosen token: 1584  ('even')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even'

Step 13 - j = 50
 token                        p_LM     xi      p_WM

when                        0.1881  0.4569  0.0737
though                      0.1371  0.6422  0.1878
without                     0.1288  0.5675  0.0583
if                          0.0987  0.8235  0.6642
in                          0.0696  0.2894  0.0000
the                         0.0389  0.7463  0.0026
.                           0.0281  0.4982  0.0000
with                        0.0190  0.3949  0.0000
before                      0.0168  0.0092  0.0000
while                       0.0163  0.5370  0.0000
 → Chosen token:  565  ('if')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if'

Step 14 - j = 51
 token                        p_LM     xi      p_WM

the                         0.3261  0.3363  0.0298
it                          0.3091  0.2754  0.0130
you                         0.0737  0.2147  0.0000
```

```
that                        0.0389  0.0529  0.0000
someone                     0.0293  0.0739  0.0000
this                        0.0267  0.0245  0.0000
they                        0.0203  0.9805  0.3194
there                       0.0174  0.8568  0.0001
its                         0.0124  0.9528  0.0170
a                           0.0097  0.9931  0.4121
 → Chosen token:  263  ('a')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a'

Step 15 - j = 52
 token                        p_LM     xi      p_WM

third                       0.1508  0.5309  0.1289
file                        0.1174  0.5721  0.0736
ha                          0.1084  0.5504  0.0348
person                      0.1042  0.6590  0.1566
user                        0.0551  0.4802  0.0000
computer                    0.0287  0.2373  0.0000
mal                         0.0267  0.9193  0.3676
security                    0.0171  0.7249  0.0000
ste                         0.0168  0.0745  0.0000
program                     0.0152  0.0050  0.0000
 → Chosen token: 4439  ('mal')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a mal'

Step 16 - j = 53
 token                        p_LM     xi      p_WM

icious                      0.7261  0.7160  0.7970
ware                        0.2686  0.6121  0.2030
war                         0.0016  0.7368  0.0000
ici                         0.0008  0.8824  0.0000
function                    0.0006  0.9768  0.0000
ice                         0.0004  0.1223  0.0000
formed                      0.0002  0.0271  0.0000
ic                          0.0002  0.7678  0.0000
-                           0.0002  0.1076  0.0000
p                           0.0001  0.7357  0.0000
 → Chosen token: 14803  ('icious')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a malicious'

Step 17 - j = 54
 token                        p_LM     xi      p_WM
```

```
user                        0.1414  0.8372  0.3248
program                     0.0854  0.6036  0.0031
attack                      0.0773  0.3845  0.0000
person                      0.0738  0.0837  0.0000
software                    0.0709  0.7898  0.0410
party                       0.0538  0.9550  0.4845
actor                       0.0499  0.8823  0.0928
code                        0.0450  0.2514  0.0000
file                        0.0409  0.8820  0.0531
entity                      0.0386  0.5626  0.0000
 → Chosen token: 6263  ('party')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a malicious party'

Step 18 - j = 55
 token                        p_LM     xi      p_WM

is                          0.1751  0.2536  0.0006
has                         0.1263  0.2013  0.0000
tries                       0.1072  0.9472  0.9346
can                         0.0507  0.6243  0.0001
knows                       0.0400  0.8242  0.0123
attempts                    0.0368  0.6832  0.0000
does                        0.0248  0.3643  0.0000
or                          0.0234  0.3882  0.0000
wants                       0.0216  0.5516  0.0000
would                       0.0205  0.0955  0.0000
 → Chosen token: 14335  ('tries')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a malicious party tries'

Step 19 - j = 56
 token                        p_LM     xi      p_WM

to                          0.9826  0.2564  0.2145
.                           0.0016  0.1998  0.0000
hard                        0.0015  0.0973  0.0000
his                         0.0010  0.9888  0.0000
their                       0.0009  0.9999  0.7674
,                           0.0006  0.9978  0.0181
it                          0.0005  0.2482  0.0000
and                         0.0004  0.0998  0.0000
hack                        0.0004  0.9176  0.0000
its                         0.0004  0.2282  0.0000
 → Chosen token: 1009  ('their')
 Current prompt so far → 'Steganography is the practice of hiding information or
data inside another file or software even if a malicious party tries their'
---  END VERBOSE GENERATION  ---
```

```
Final partial generation:
 Steganography is the practice of hiding information or data inside another file
or software even if a malicious party tries their
```